

FinanceScope

Plano tecnico e mapa visual de desenvolvimento

Projeto beta para GitHub, entrevistas e evolucao tecnica
Junho de 2026

1. Visao geral

FinanceScope e um sistema web desktop-first de gestao financeira pessoal com foco em dashboard, diagnostico e simulacao de decisoes. O objetivo do projeto beta e provar o diferencial RealCost: traduzir gastos em horas de trabalho, impacto na renda, risco de fechamento do mes e atraso em metas.

O nome FinanceScope deve ser usado no repositorio, no README, nos prints, nos commits de apresentacao e neste plano. O nome final do site/produto permanece como decisao futura, sem bloquear o MVP.

2. Melhorias sobre o escopo original

- Padronizar o nome como FinanceScope para evitar divergencia entre PDF, GitHub e apresentacao.
- Enxugar o MVP para priorizar perfil financeiro, transacoes, dashboard, metas, simulador e RealCost Engine.
- Mover login, score financeiro, assinaturas avancadas, relatorios completos, importacao CSV e exportacao PDF para roadmap futuro.
- Tratar o dashboard como ferramenta de decisao, nao apenas como painel de graficos.
- Exibir formulas e explicacoes simples para aumentar confianca e demonstrar maturidade tecnica.
- Usar linguagem de estimativa educacional, evitando recomendacao financeira personalizada.
- Adicionar campos de auditoria e padronizacao no modelo de dados para facilitar evolucao.

3. Mapa visual das fases

Fase	Entrega	Resultado esperado
1	Base do projeto e identidade visual	Estrutura Flask, templates, CSS base, layout desktop e navegacao principal.
2	Perfil financeiro e dados iniciais	Usuario define renda, horas mensais, limite do mes e meta principal.
3	CRUD de transacoes	Receitas e despesas podem ser criadas, editadas, filtradas e excluidas.
4	Dashboard e graficos	Cards, categorias, evolucao mensal, ranking de gastos e alertas basicos funcionando.
5	RealCost Engine	Gastos convertidos em horas, percentual da renda, risco e atraso de meta.
6	Metas e simulador	Usuario acompanha objetivos e testa compras antes de registrar.
7	Polimento e GitHub	README, prints, dados demo, testes, deploy e apresentacao final.

4. Regras funcionais do MVP

Modulo	Regras obrigatorias
Perfil financeiro	Guardar renda mensal, horas mensais trabalhadas/estudadas, limite planejado e meta principal.
Transacoes	Permitir tipo receita/despesa, descricao, valor positivo, data, categoria, forma de pagamento e recorrencia simples.
Dashboard	Mostrar receitas, despesas, saldo, previsao de fechamento, maior vazamento, graficos e alertas.
Metas	Criar meta com valor alvo, valor atual e aporte mensal; calcular progresso e meses estimados.
Simulador	Receber item, valor, categoria, prioridade e parcelas; retornar impacto antes da decisao.
RealCost	Centralizar formulas em utilitario separado e reutilizar no dashboard, simulador e detalhes de despesa.
Alertas	Classificar risco como baixo, medio ou alto com explicacao curta e nao proibitiva.

5. Regras de calculo

Calculo	Formula	Uso no produto
Valor da hora	$\text{renda_mensal} / \text{horas_mensais}$	Base para traduzir compras em tempo de esforco.
Custo em horas	$\text{valor} / \text{valor_da_hora}$	Exibir em horas e minutos no simulador e nos detalhes.
Impacto na renda	$(\text{valor} / \text{renda_mensal}) * 100$	Mostrar peso percentual de compras e categorias.
Saldo do mes	$\text{total_receitas} - \text{total_despesas}$	Card principal do dashboard.
Previsao de fechamento	$\text{media_diaria_despesas} * \text{dias_do_mes}$	Alerta de risco quando ultrapassar limite planejado.
Tempo da meta	$(\text{valor_alvo} - \text{valor_atual}) / \text{aporte_mensal}$	Estimativa de meses para concluir objetivo.
Atraso da meta	$\text{valor_compra} / \text{aporte_mensal}$	Quantos meses de aporte a compra consome.
Custo anual recorrente	$\text{valor_mensal} * 12$	Roadmap para modulo de assinaturas.

Se renda mensal, horas mensais ou aporte mensal forem zero ou vazios, o sistema deve evitar divisao por zero e orientar o usuario a completar o perfil financeiro.

6. Arquitetura tecnica

Camada	Responsabilidade
Templates HTML	Estruturar telas, formularios, tabelas e componentes principais.
CSS	Layout SaaS desktop-first, sidebar, cards, responsividade e estados visuais.
JavaScript	Interacoes, validacoes leves, Chart.js e atualizacoes de interface.
Flask	Rotas, processamento de formularios, regras de negocio e integracao com banco.
SQLite	Persistencia local de perfil, transacoes, metas, categorias e simulacoes.
utils/finance.py	Formulas do RealCost, score futuro e funcoes de previsao.
tests/	Testes das formulas e dos fluxos principais.

7. Modelo de dados recomendado

Tabela	Campos principais
users	id, name, monthly_income, monthly_hours, monthly_limit, created_at, updated_at
categories	id, name, type, color, is_default, created_at
transactions	id, user_id, type, description, category_id, amount, date, payment_method, is_recurring, created_at, updated_at
goals	id, user_id, name, target_amount, current_amount, monthly_contribution, priority, created_at, updated_at
purchase_simulations	id, user_id, item_name, amount, category_id, installments, risk_level, result_json, created_at

Padronizacoes recomendadas: type deve aceitar apenas income ou expense; valores monetarios devem ser sempre positivos; recorrência deve ser booleana no MVP e pode virar tabela propria em versao futura.

8. Critérios de aceite por fase

Fase	Aceite
1	A aplicacao abre localmente, possui layout consistente, sidebar e telas principais navegaveis.
2	Perfil salva renda, horas mensais, limite e meta principal; validacoes impedem valores invalidos.
3	CRUD de transacoes persiste em SQLite e filtros por mes, categoria e tipo funcionam.
4	Dashboard exibe cards corretos, dois graficos Chart.js e ranking dos maiores gastos.
5	RealCost calcula horas, impacto percentual, risco e atraso de meta com testes unitarios.
6	Metas e simulador usam dados reais do perfil e mostram mensagens claras de decisao.
7	README completo, dados demo, prints, instrucoes de execucao, deploy e revisao final.

9. Checklist final para GitHub

- README com problema, solucao, diferencial RealCost, stack, funcionalidades e prints.
- Secao Como rodar com ambiente virtual, dependencias, inicializacao do banco e comando Flask.
- Dados de demonstracao para apresentar dashboard, graficos, metas e simulador sem cadastro manual demorado.

- GIF curto mostrando cadastro de despesa e resultado do simulador Posso Comprar?.
- Aviso educacional: as analises sao estimativas baseadas nos dados informados e nao substituem planejamento financeiro profissional.
- Issues ou roadmap com login, importacao CSV, assinaturas completas, relatorios PDF, API externa e responsividade mobile.
- Commits organizados por fase para demonstrar processo de desenvolvimento.

10. Backlog recomendado

Prioridade	Item	Motivo
MVP	Perfil, transacoes, dashboard, RealCost, metas e simulador	Produto diferencial com menor escopo viavel.
Pos-MVP	Score financeiro e assinaturas completas	Aumenta profundidade apos dados centrais estarem estaveis.
Pos-MVP	Relatorios por periodo e exportacao PDF	Ajuda apresentacao e evolucao tecnica.
Futuro	Login e multiplos usuarios	Necessario para produto real, mas nao para demonstracao inicial.
Futuro	Importacao CSV e integracao bancaria	Eleva complexidade e deve vir apenas depois da base validada.

11. Regra de comunicacao do produto

O FinanceScope deve ajudar o usuario a decidir, nao mandar nele. As mensagens devem ser objetivas, explicaveis e transparentes. Exemplo: 'Esta compra equivale a 18h40 de trabalho e pode atrasar sua meta em 1,2 mes, considerando os dados informados no perfil.'

- Use 'estimativa', 'pode indicar', 'com base nos seus dados' e 'no ritmo atual'.
- Evite frases como 'nao compre', 'voce deve investir' ou 'garantia de economia'.
- Mostre sempre a formula ou uma explicacao simples perto do resultado principal.

12. Definicao de pronto do MVP

1. Usuario configura perfil financeiro e visualiza dados no dashboard.
2. Usuario cadastra, edita, filtra e remove transacoes.
3. Dashboard mostra receitas, despesas, saldo, categorias, evolucao e alertas.
4. RealCost Engine tem testes automatizados para os calculos principais.
5. Usuario cria meta e o sistema calcula progresso e prazo estimado.
6. Simulador mostra impacto de compra em horas, renda, risco e meta.
7. Projeto possui README, dados demo, prints, aviso educacional e instrucoes de execucao.